

LEARNING FROM OTHERS, ACTING FOR YOU: CROSS-USER MEMORY SHARING FOR LLM AGENTS

Jinming Hu^{1*}, Haodong Zhao^{1,3*}, Qi Jia^{2†}, Die Chen³, Tianhang Zhao¹,
Sufeng Duan¹, Gongshen Liu^{1†}

¹School of Computer Science, Shanghai Jiao Tong University

²Shanghai Artificial Intelligence Laboratory

³School of Computing, National University of Singapore

{hujinming, zhaohaodong}@sjtu.edu.cn, jiaqi@pjlab.org.cn,
daphnechen@u.nus.edu, {zthzthzth, 1140339019dsf, lgshen}@sjtu.edu.cn

ABSTRACT

Large language model (LLM) agents serving different users often solve related tasks, yet separate user histories can leave reusable experience inaccessible to other agents. Pooling memories expands access but risks transferring preferences that conflict with the receiving user’s requirements. We introduce ShareMem, a memory architecture that shares reusable experience while grounding its application in the receiving user’s own preferences. Shared experiences indicate how to act and which preferences to consult; the receiving user’s memory supplies their concrete values. Two-stage consolidation refines experience locally before integrating accepted edits into a shared pool. During execution, scope-first retrieval jointly selects local and shared experiences under a common entry budget, while a user-bound channel supports initial and agent-initiated preference retrieval. We evaluate ShareMem across web navigation (Mind2Web), online personalized interaction (VitaBench 2.0), and multi-session coding (MemoryCode) with four backbone models. It improves step success, average task success, and dialogue-macro coding scores, respectively, over matched user-local memory across all four models. Ablations favor two-stage consolidation for smaller shared pools, lower induction token usage, and better downstream performance, and support complementarity between experience guidance and active preference retrieval. Further analyses show that sharing helps most when relevant local experience is scarce, while source quality and cross-user preference interference limit useful transfer.

1 INTRODUCTION

Long-term memory enables large language model (LLM) agents to reuse past experience and personalize assistance (Park et al., 2023; Packer et al., 2023; Zhong et al., 2024; Chhikara et al., 2025). Consider an organization where employees use the same agent system, each with a personalized instance and a separate interaction history. These histories contain reusable procedures for common tasks alongside personal preferences and requirements. Experience acquired by one instance may therefore benefit others, provided that its application respects the receiving user’s constraints.

Keeping these histories local preserves their association with individual users, but also isolates useful experience: an agent may lack guidance already acquired by another for a similar task. We call this an *experience-reachability bottleneck*. Pooling histories expands access, but can introduce instructions that conflict with the receiving user’s requirements. For example, a coding procedure may be reusable across employees, while the contributor’s preferred naming convention may not be.

Existing works distill interaction histories into lessons, skills, or workflows for subsequent tasks (Shinn et al., 2023; Zhao et al., 2024; Wang et al., 2024b). Shared repositories, collaborative memory, and federated skill learning extend reuse across agents through experience aggregation,

*These authors contributed equally.

†Corresponding authors.

access control, and collective refinement (Gao & Zhang, 2024; Rezazadeh et al., 2025; Ma et al., 2026; Yang et al., 2026). However, making experience accessible does not establish its applicability to another user. Access permissions govern retrieval, while successful personalization also requires separating reusable procedures from user-specific preferences. This motivates our central question: *How can agents reuse others’ experience while applying their own user preferences?*

We introduce ShareMem, a memory architecture that shares guidance about how to perform tasks and which personal constraints to consult, while resolving concrete values through the receiving user’s own memory. Each agent maintains local experiences and personal preferences. Two-stage consolidation extracts reusable local experience and consolidates accepted edits in a shared pool. During execution, scope-first retrieval jointly selects local and shared experiences under a joint entry budget, while a user-bound preference channel supports initial and agent-initiated recall of personal constraints. Across web navigation, online personalized interaction, and multi-session coding with four backbone models, comparisons with matched user-local configurations show gains from sharing, especially where relevant local experience is sparse. Further analyses examine consolidation and active preference retrieval, and identify source quality and cross-user preference interference as limits to useful transfer. Our contributions are:

Experience-reachability perspective. We identify the isolation of reusable experience as a bottleneck for personalized agents, distinguishing access to useful procedures from the applicability of user-specific values.

Selective memory sharing architecture. We introduce ShareMem, which combines two-stage experience consolidation and joint local–shared retrieval with a separate preference channel bound to the receiving user.

Empirical characterization of transfer. We characterize when sharing helps and where it falls short, examining local coverage, memory consolidation, retrieval behavior, and interference from other users’ preferences.

2 RELATED WORK

Individual agent memory and personalization. Generative Agents integrates experience streams with reflection and retrieval (Park et al., 2023), while CoALA places memory within agent cognition (Sumers et al., 2023). MemGPT, MemoryBank, Mem0, and A-Mem develop long-term storage and retrieval (Packer et al., 2023; Zhong et al., 2024; Chhikara et al., 2025; Xu et al., 2026), and personalized memory systems adapt assistance through user-grounded retrieval and interaction histories (Salemi et al., 2024; Wang et al., 2024a; Tan et al., 2025). Building on this user-specific grounding, we study how agents can also benefit from experience outside their own histories.

Abstracting and maintaining experience. Beyond retaining histories, experience abstraction produces reusable guidance. Reflexion and ExpeL extract feedback and lessons (Shinn et al., 2023; Zhao et al., 2024), while Voyager and Agent Workflow Memory develop reusable skills and workflows (Wang et al., 2023; 2024b). ReasoningBank, Agentic Context Engineering, and MCMA explore abstraction, refinement, and maintenance (Ouyang et al., 2026; Zhang et al., 2026; Liang et al., 2026). When experience is shared across users, redundant contributions can compete for limited retrieval slots. ShareMem addresses this through two-stage consolidation: refining local experience before integrating accepted edits into a shared pool.

Sharing experience across users and agents. Memory Sharing aggregates agent experience (Gao & Zhang, 2024), while Collaborative Memory manages multi-user sharing under asymmetric access control (Rezazadeh et al., 2025). SkillClaw, FederatedSkill, and lifelong multi-agent memory systems study collective skill evolution and experience reuse (Ma et al., 2026; Yang et al., 2026; Wu et al., 2026). Agent KB and FedWorld examine knowledge applicability across environments or clients (Tang et al., 2025; Hou, 2026). Retrieval permission and contextual relevance alone do not establish whether a preference applies to the receiving user. ShareMem therefore pairs scope-first experience retrieval with initial and active recall from that user’s own memory, separating transferable guidance from personal constraints. Our interference analysis examines this boundary.

3 SHAREMEM: SELECTIVE MEMORY SHARING

ShareMem enables agents to reuse experience across users while grounding its application in the receiving user’s preferences. Figure 1 summarizes two-stage experience consolidation, scope-first local-shared retrieval, and a separate preference channel supporting initial and active recall.

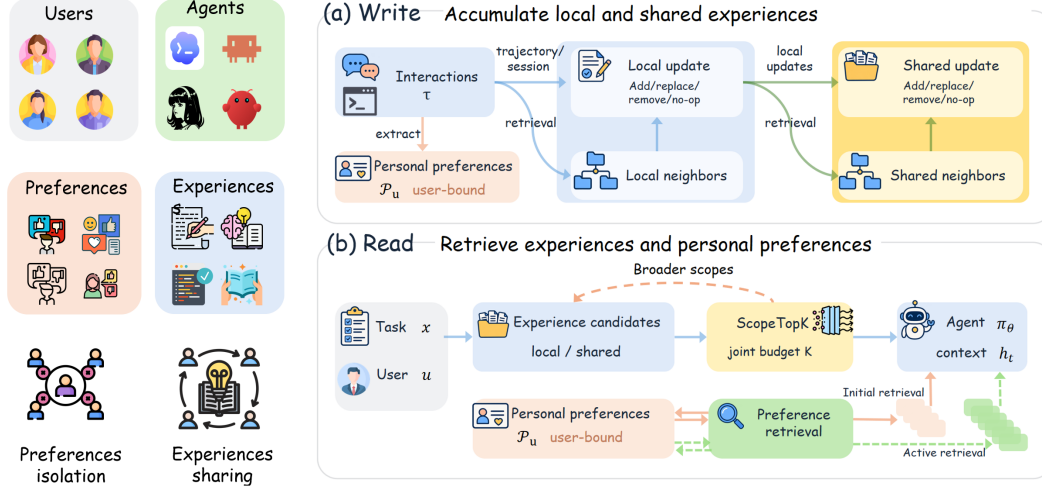


Figure 1: **ShareMem overview.** Left: agents share reusable experiences while keeping preferences user-bound. (a) Two-stage consolidation: interactions and retrieved local experiences inform local updates; accepted edits then provide evidence for consolidation with shared experiences. (b) Scope-first retrieval selects local and shared experiences under a joint budget K . A separate channel supports initial and agent-initiated retrieval of the receiving user’s preferences.

3.1 MEMORY STRUCTURE

We consider a multi-user deployment, such as an organization or a shared agent platform, where each user u has a dedicated agent instance A_u and interaction history $\mathcal{H}_u$. Users may face similar tasks while differing in their preferences and requirements.

Local memory $\mathcal{M}_u$ comprises preferences $\mathcal{P}_u = \{p_u^1, \dots, p_u^{n_u}\}$ and experiences $\mathcal{E}_{u,s} = \{e_{u,s}^1, \dots, e_{u,s}^{m_{u,s}}\}$ for applicability scope s . Here n_u and $m_{u,s}$ are current collection sizes. The i th preference p_u^i records a personal requirement; the i th experience $e_{u,s}^i$ describes a reusable procedure and relevant preference dimensions. Shared memory $\mathcal{S}$ comprises cross-user experience collections $\mathcal{R}_s$ under the same scopes. An environment-specific adapter assigns scopes and retrieval priorities.

Preferences are maintained from each user’s history separately from experience consolidation (Appendix A.3). During execution, agents retrieve procedural guidance from local and shared experiences and resolve personal requirements through the receiving user’s $\mathcal{P}_u$. Thus, sharing expands experience access without granting access to other users’ preference stores.

3.2 TWO-STAGE EXPERIENCE CONSOLIDATION

For a trajectory or session $\tau \in \mathcal{H}_u$ eligible under the evaluation protocol, let $s = \sigma(\tau)$, where σ is the adapter’s scope-assignment function. The local retriever R_{local} collects up to k_w relevant neighbors:

$$\mathcal{N}_{u,s} = R_{\text{local}}(\tau, \mathcal{E}_{u,s}; k_w). \quad (1)$$

The LLM-based update operator U_{local} uses this evidence and neighborhood to edit the collection:

$$(\mathcal{E}'_{u,s}, \Delta_{u,s}) = U_{\text{local}}(\mathcal{E}_{u,s}, \tau; \mathcal{N}_{u,s}), \quad (2)$$

Primes denote updated collections. The edit set $\Delta_{u,s}$ contains accepted additions, replacements, or removals; a no-op leaves it empty. The manager is instructed to abstract source-specific values into

semantic slots and preference-retrieval guidance. Retrieved neighbors support consolidation with existing procedures; an empty neighborhood still permits additions.

For each edit $\delta \in \Delta_{u,s}$, R_{shared} retrieves up to k_w shared neighbors. Their union supplies the shared update operator U_{shared} :

$$\mathcal{N}_s^{\text{shared}} = \bigcup_{\delta \in \Delta_{u,s}} R_{\text{shared}}(\delta, \mathcal{R}_s; k_w), \quad (3)$$

$$\mathcal{R}'_s = U_{\text{shared}}(\mathcal{R}_s, \Delta_{u,s}; \mathcal{N}_s^{\text{shared}}). \quad (4)$$

The shared manager follows the same abstraction rules but decides edits against its own collection. A local replacement or removal need not induce the same shared operation; without accepted local edits, shared memory is unchanged.

3.3 JOINT EXPERIENCE RETRIEVAL AND ACTIVE PREFERENCE RETRIEVAL

Scope-first experience retrieval. For task x from user u , the adapter constructs J scope groups $\mathbf{G}(x) = (G_0, \dots, G_{J-1})$, from direct matches to broader fallbacks. Each G_j contains scopes at priority j , with smaller j indicating closer matches. Its local-shared candidate set is

$$C_j = \bigcup_{s \in G_j} [R(x, \mathcal{E}_{u,s}; K_c) \cup R(x, \mathcal{R}_s; K_c)], \quad (5)$$

where the experience retriever R returns up to K_c candidates per collection. The guidance set H_x is

$$H_x = \text{ScopeTopK}(x, (C_0, \dots, C_{J-1}), K). \quad (6)$$

Within each group, ScopeTopK jointly reranks both sources under a single K -entry budget. Lower-priority groups only fill remaining slots and cannot displace earlier selections. Selection stops at K entries or candidate exhaustion. Appendix A.3 specifies benchmark-specific candidate filtering and duplicate handling.

Active preference retrieval. For tasks with a preference channel, $R_{\text{pref}}(q, \mathcal{P}_u; k_p)$ retrieves at most k_p entries for query q . Initial recall $B_u^0 = R_{\text{pref}}(x, \mathcal{P}_u; k_p)$ joins x and H_x in message context h_0 . At step t , LLM policy π_θ with parameters θ selects action a_t from context h_t :

$$a_t \sim \pi_\theta(\cdot \mid h_t), \quad a_t \in \mathcal{A}_{\text{env}} \cup \mathcal{A}_{\text{pref}} \cup \mathcal{A}_{\text{out}}, \quad (7)$$

The action spaces represent environment interaction, preference retrieval, and terminal responses, respectively. The policy decides whether and what to retrieve; experiences guide this decision without automatically triggering it. A preference action $a_t \in \mathcal{A}_{\text{pref}}$ with query q_t returns observation

$$o_t = R_{\text{pref}}(q_t, \mathcal{P}_u; k_p). \quad (8)$$

The environment fixes u , preventing cross-user preference access. Read-only retrieval appends the action and its observation to the context:

$$h_{t+1} = h_t \parallel (a_t, o_t), \quad (9)$$

where $\parallel$ denotes context concatenation. The agent can retrieve again or act within the execution budget. Experiences guide which constraints to resolve, while current task requirements and personal preferences take precedence.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Benchmarks and metrics. **Mind2Web** evaluates instruction-following web agents on real-world websites, with human-demonstrated action sequences and generalization across tasks, websites, and domains (Deng et al., 2023). We report task-macro element accuracy, action F1, and step success, measuring element selection, action prediction, and their joint correctness. **VitaBench 2.0** evaluates personalized and proactive behavior in temporally ordered user interactions across delivery, in-store consumption, and travel services (Chen et al., 2026). Tasks require tracking evolving preferences and

acquiring missing information, with rubric-based evaluation of task execution. We report Avg@3, Pass@3, and Pass³: mean success over three trials, success in at least one trial, and success in all three, respectively. All scores use the same 771 subtasks; success requires full reward, and missing or error outcomes count as failures. **MemoryCode** is a multi-session coding benchmark that tests whether models retain and apply coding instructions amid irrelevant information and changing requirements (Rakotonirina et al., 2025). Generated code is assessed against the applicable rules using the original AST/regex evaluator. We report dialogue-macro reward on a 0–100 scale, denoted D-MS.

Baselines. We construct five controlled memory configurations. NONE disables long-term memory; PREF uses only the current user’s personal preferences. LOCAL adds user-local reusable experiences. SHARE-ALL additionally shares experiences and retrieves other users’ preferences as source-labeled, non-authoritative hints, reserving slots for the current user. SHAREMEM shares reusable experiences while keeping preference retrieval user-bound; LOCAL is its counterpart without cross-user sharing. Mind2Web omits PREF and SHARE-ALL because it has no preference channel. We additionally compare with benchmark-specific references: AWM (Wang et al., 2024b) on Mind2Web; RAG, Rewrite, Full-CTX, and oracle Ground-truth on VitaBench 2.0; and Full-CTX and Full-Pref on MemoryCode.

Models. We use Qwen 3.6-27B (Qwen Team, 2026), DeepSeek v4-flash (DeepSeek-AI, 2026), Gemma 4-26B (Gemma Team, 2026), and Gemini 3 Flash (Google DeepMind, 2025) as foundation models. Retrieval uses BGE-M3 embeddings and BGE-Reranker-v2-M3 (Li et al., 2023; Chen et al., 2024).

Implementation details. Scopes are supplied by website metadata in Mind2Web, task domains in VitaBench 2.0, and a single coding scope in MemoryCode. Mind2Web and MemoryCode construct experience collections offline from designated training trajectories or historical sessions and keep them fixed during evaluation. VitaBench 2.0 starts with empty experience collections and updates them online from completed full-reward subtasks, making accepted updates available to subsequent tasks. We set $k_w = 5$, $K = 5$, $k_p = 10$, and $K_c = 25$. Appendix A details benchmark protocols and score aggregation; Appendices A.2 and A.3 specify memory-access configurations and update/retrieval procedures, respectively.

4.2 MAIN RESULTS

Table 1: **Mind2Web.** Three-seed task-macro scores (%). EA: element accuracy; Act. F1: action F1; Step: step success. Δ : SHAREMEM – LOCAL (pp). Bold marks the best score.

Memory	Qwen 3.6-27B			DeepSeek v4-flash			Gemma 4-26B			Gemini 3 Flash		
	EA	Act. F1	Step	EA	Act. F1	Step	EA	Act. F1	Step	EA	Act. F1	Step
None	42.40	48.70	36.90	42.66	50.17	36.77	39.23	47.13	32.22	43.12	49.04	38.12
LOCAL	47.07	56.81	43.06	48.12	58.35	44.17	45.22	56.68	40.67	50.14	57.13	45.29
SHAREMEM	51.32	58.63	47.16	51.80	59.35	47.96	47.05	58.20	43.13	51.87	58.35	47.64
Δ S–L	+4.25	+1.82	+4.10	+3.68	+1.00	+3.79	+1.83	+1.52	+2.46	+1.73	+1.22	+2.35

On Mind2Web, Table 1 shows that LOCAL outperforms NONE across all four backbones, demonstrating the value of accumulated experience for web interaction. ShareMem further improves all three metrics, increasing step success by 2.35–4.10 percentage points over LOCAL and by approximately 10.5 points over NONE on average across backbones. The accompanying gains in element accuracy and action F1 indicate improvements in both webpage-element selection and action prediction. Since this evaluation excludes same-site experience from the receiving user’s local history, the gains over LOCAL support the value of accessing relevant experience from other users. Section 4.4 examines how this benefit changes as local experience coverage increases.

On VitaBench 2.0, Table 2 shows that access to personal preferences provides the largest gain over NONE, whereas adding local experiences yields only small, inconsistent changes. Sharing experience provides a more consistent additional benefit: ShareMem achieves the highest Avg@3 across

Table 2: **VitaBench 2.0**. Subtask-level scores (%) over three trials. Bold marks the best score; [†] marks a deficit to SHARE-ALL below 0.5 pp. Δ : SHAREMEM – LOCAL (pp).

	Qwen 3.6-27B			DeepSeek v4-flash			Gemma 4-26B			Gemini 3 Flash		
Condition	Avg@3	Pass@3	Pass ³	Avg@3	Pass@3	Pass ³	Avg@3	Pass@3	Pass ³	Avg@3	Pass@3	Pass ³
None	6.96	13.75	1.69	7.05	12.97	2.33	5.53	10.89	1.82	8.69	15.43	3.24
PREF	26.93	46.82	9.99	22.78	41.50	7.65	17.51	34.76	3.89	20.88	42.41	4.41
LOCAL	26.93	47.21	9.34	22.57	39.95	7.65	18.33	36.71	3.37	21.88	44.10	5.06
SHARE-ALL	25.90	46.04	8.43	23.99	42.80	7.52	18.24	36.84	4.41	22.96	42.93	5.84
SHAREMEM	29.10	49.94	11.28	25.03	42.67 [†]	9.08	19.67	39.82	4.02 [†]	23.13	45.01	4.93
Δ S–L	+2.16	+2.72	+1.95	+2.46	+2.72	+1.43	+1.34	+3.11	+0.65	+1.25	+0.91	−0.13

all four backbones, outperforming LOCAL by approximately 1.8 percentage points on average. Extending access to other users’ preferences does not improve this average performance, as SHARE-ALL trails ShareMem on Avg@3 for every backbone. This advantage, however, does not hold uniformly across metrics: SHARE-ALL achieves the highest Pass@3 for DeepSeek and Pass³ for Gemma and Gemini, while ShareMem falls slightly below LOCAL on Gemini’s Pass³. Thus, sharing improves average task success more consistently than repeatability.

Table 3: **MemoryCode**. Mean D-MS (0–100) over three fixed 20-dialogue subsets. Bold marks the best mean; Δ : SHAREMEM – LOCAL (pp). The benchmark protocols and aggregation subsection of Appendix A specifies subset construction and dialogue-macro scoring.

Backbone	None	PREF	LOCAL	SHARE-ALL	SHAREMEM	Δ S–L
Qwen 3.6-27B	2.58	59.84	66.41	67.38	69.32	+2.91
DeepSeek v4-flash	1.39	77.46	78.37	77.21	78.53	+0.16
Gemma 4-26B	2.14	56.91	56.24	50.05	57.63	+1.39
Gemini 3 Flash	1.49	64.12	68.07	62.98	69.01	+0.94

On MemoryCode, Table 3 shows a large improvement from NONE to PREF, highlighting the importance of access to personal coding requirements. Sharing reusable experience provides an additional benefit: ShareMem achieves higher mean D-MS than LOCAL across all four backbones, although the gain for DeepSeek is marginal. Broader preference access is less effective: SHARE-ALL scores below ShareMem for every backbone and below PREF for Gemma and Gemini. Together, these results support sharing reusable experience while keeping preference retrieval bound to the receiving user’s own coding requirements.

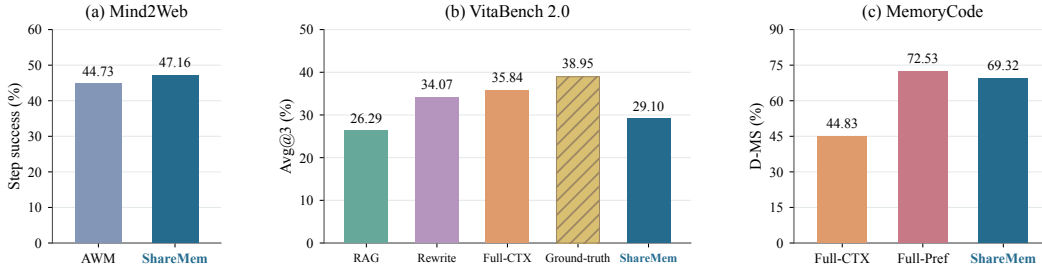


Figure 2: **Benchmark-specific reference comparisons with Qwen**. Three-run means. Full-CTX supplies raw interaction history; Full-Pref supplies all extracted personal preferences without shared experiences or active retrieval.

Figure 2 complements the controlled comparison with LOCAL by evaluating ShareMem against benchmark-specific reference systems. On Mind2Web, ShareMem outperforms AWM (Wang et al., 2024b) by 2.43 percentage points in step success. On VitaBench 2.0, it exceeds RAG with BGE-M3 retrieval but trails history rewriting and full-context conditioning, indicating that selective access does not capture all the benefits of richer historical context. Ground-truth supplies oracle preferences

as an additional reference. On MemoryCode, ShareMem substantially outperforms raw-history conditioning but remains 3.21 D-MS points below access to the complete extracted preference collection. These system-level comparisons highlight both the value and the limits of selective memory: concise experiences and preferences can be more effective than raw history, while broader access to relevant context or personal requirements can still provide additional gains.

4.3 ABLATION STUDIES

We examine the contributions of experience consolidation, scope-first retrieval, and active preference retrieval. For each comparison, we clarify changes to memory construction, retrieval policy, or preference access to contextualize the observed performance differences.

Two-stage consolidation. We first examine whether consolidating local experience before sharing improves the resulting shared memory. We compare ShareMem with direct shared-memory writing, which processes raw sessions instead of accepted local edits. Both configurations retain an editable shared manager. Local pools are rebuilt across two-stage runs, so this comparison evaluates the writing procedures without holding local memory fixed. Two-stage consolidation produces a smaller shared experience pool (Figure 3) while achieving higher D-MS on all three subsets (Table 4). It also reduces shared-memory induction token usage (Table 9 in Appendix B.1). Together, these results support consolidating experience locally before sharing: the resulting shared memory combines a more compact pool and lower construction overhead with better downstream performance.

Table 4: **Write-path ablations.** Experiments conducted on MemoryCode with Qwen. D-MS (%), dialogue-macro). 2S: two-stage; D: direct; A: append-only. Pools are rebuilt for each configuration.

Subset	D-MS $\uparrow$			Global entries		
	2S	D	A	2S	D	A
42	69.34	64.42	62.74	40	67	462
23	66.94	66.75	65.70	17	79	467
7	71.67	65.68	68.60	1	82	493

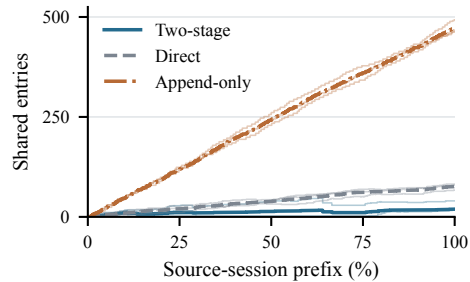


Figure 3: **Experience-pool dynamics.**

Editable consolidation. We next examine whether shared memory benefits from revising existing experience rather than continually accumulating new entries. ShareMem supports adding, replacing, and removing entries, or leaving memory unchanged. The append-only variant adds newly extracted experiences with exact-string deduplication but neither revises nor removes existing entries. It also omits the existing pool from the extraction context, so this ablation changes both experience extraction and memory maintenance. Append-only storage yields lower D-MS on all three subsets (Table 4) despite producing substantially larger shared pools (Figure 3). Under the fixed five-entry retrieval budget, larger pools may increase competition among near-duplicate candidates without providing more useful guidance. Together, these results favor pool-aware, editable consolidation for maintaining compact and effective shared memory.

Scope-first retrieval. We examine whether scope information improves experience selection beyond similarity alone. Flat search jointly ranks all accessible local and shared experiences across scopes, without scope filtering or prioritization. We compare it with scope-first retrieval using frozen Mind2Web pools, the same similarity function, and matched retrieval budgets. In the Qwen sweep, scope-first retrieval achieves higher step success across the tested budgets (Figure 4(a)). The accompanying retrieval analysis shows a consistently larger fraction of entries from the task’s website (Appendix B.2). Although this fraction decreases for both strategies as the budget grows, scope-first retrieval retains substantially higher same-site coverage. Together, these results support scope priority as a way to improve contextual alignment with corresponding performance gains.

Active preference retrieval. We test whether experience guidance and active preference retrieval provide complementary benefits on MemoryCode. With experience pools frozen, we compare initial recall alone with initial recall plus agent-initiated retrieval under two conditions: PREF, which

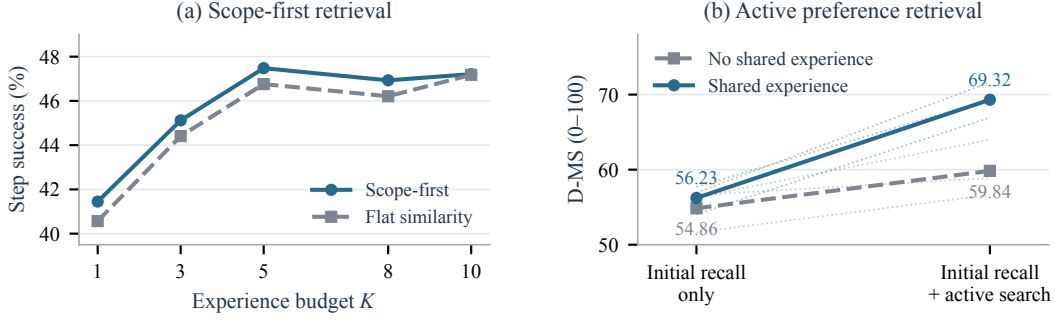


Figure 4: **Retrieval ablations.** (a) Mind2Web, scope-first retrieval outperforms flat retrieval across different experience budgets K . (b) Qwen, MemoryCode: three-subset means and dotted subset curves compare preference-only access with local and shared experience guidance, under initial recall alone or initial plus active recall.

provides only personal preferences, and ShareMem, which additionally provides local and shared experience guidance. Relative to PREF, ShareMem improves mean D-MS by 9.48 percentage points with active retrieval, compared with 1.37 points under initial recall alone (Figure 4(b)). This larger gain holds across all three subsets, supporting complementarity between reusable guidance and access to user-specific constraints during execution. The comparison measures the combined contribution of local and shared experience, rather than the incremental benefit of sharing alone. Moreover, disabling active retrieval restricts both information access and interaction opportunities, so these results do not isolate retrieval frequency as the source of improvement.

4.4 FURTHER ANALYSIS

Local experience coverage. We examine how the benefit of sharing changes with the availability of relevant local experience. Keeping the Mind2Web training and evaluation tasks fixed, we vary their assignment to users. Exclusive assigns each website’s training trajectories to one user and its evaluation tasks to another. Task-uniform assigns tasks through a seeded hash, while Site-balanced distributes them round-robin within each website using a shared user offset across splits. Across three seeds, these assignments produce mean same-site local coverage of 0.00%, 49.60%, and 98.81%, respectively (Table 12). As coverage increases, the corresponding mean gains of ShareMem over LOCAL in task-macro step success decline from 4.10 to 1.30 and -0.13 percentage points (Figure 5(a)). These results show that sharing is most useful in this setting when it supplies relevant experience missing from the recipient’s history; its incremental benefit largely disappears when that experience is already available locally.

Experience quality. We next examine sensitivity to evidence quality by replacing gold demonstrations with cached agent rollouts while holding training-task coverage fixed. This intervention changes the evidence used for both experience construction and attached demonstrations. Moving from all-gold to all-rollout evidence reduces mean task-macro step success by 5.90 percentage points for ShareMem, compared with 1.53 points for LOCAL, across three seeds (Figure 5(b)). The decline in ShareMem holds in every seed. Under this split, LOCAL retrieves only cross-site experience, whereas shared guidance is usually same-site; the smaller LOCAL decline therefore does not establish greater robustness to unreliable evidence. The asymmetry is consistent with source quality becoming more consequential when sharing supplies experience relevant to the current task. These results highlight the need to consider evidence reliability alongside experience coverage when constructing shared memory.

Admission thresholds. We next examine how admission filtering balances rollout quality against experience coverage. In cold-start online Mind2Web, agents begin with empty experience pools and update them from rollouts on evaluation tasks; admitted experience becomes available to subsequent tasks. A rollout is admitted only when its step-success rate reaches τ , with $\tau \geq 0.5$ in this comparison. Stricter thresholds slow local experience accumulation (Figure 6). As τ increases from 0.5 to 0.999, LOCAL loses approximately 2.6 percentage points in step success, while ShareMem

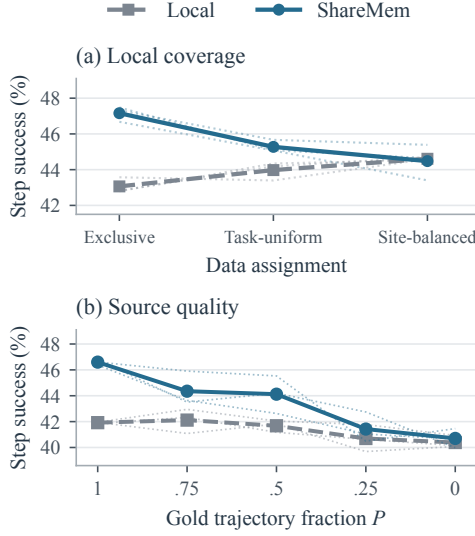


Figure 5: **Experience coverage and quality.** (a) Step success under different user assignments; thick curves show three-seed means and faint dotted curves individual seeds. (b) Replacing gold demonstrations with cached agent rollouts at fixed training-task coverage.

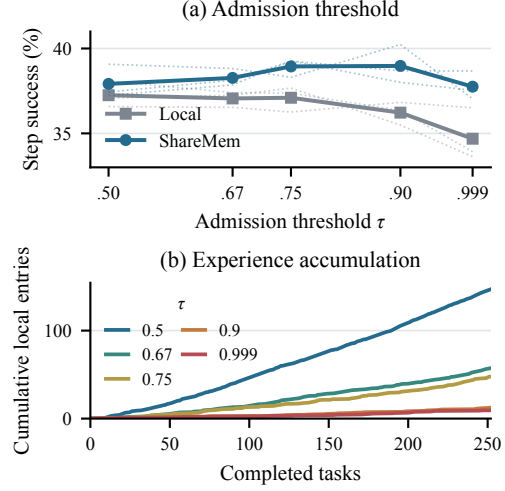


Figure 6: **Admission and accumulation.** Qwen, cold-start online Mind2Web; three-seed means. (a) Task-macro step success versus admission threshold τ ; dotted curves show individual seeds. (b) Total local experience entries across users versus completed tasks.

changes less overall but declines from its intermediate-threshold peak. Online scores also remain substantially below the offline results, consistent with reliance on noisier agent-generated evidence and limited experience during cold start. Unlike the source-quality sweep, which preserves training-task coverage, admission filtering can exclude evidence altogether. Together, these results highlight a trade-off: stricter filtering raises the success requirement for admitted rollouts but can leave agents without useful guidance. Effective shared memory therefore requires attention to both evidence quality and experience coverage.

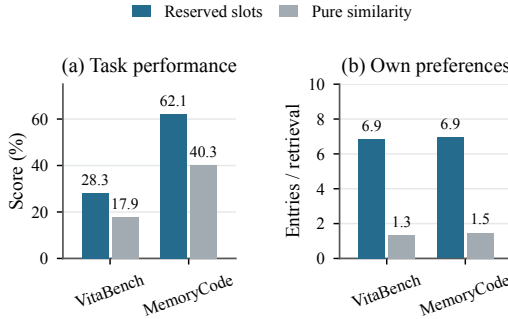


Figure 7: **Preference-slot reservation.** We compare two retrieval settings: reserved preference slots versus pure similarity ranking. (a) Strict success on VitaBench 2.0 and D-MS on MemoryCode under the two settings. (b) Mean number of current-user entries per top-10 retrieval.

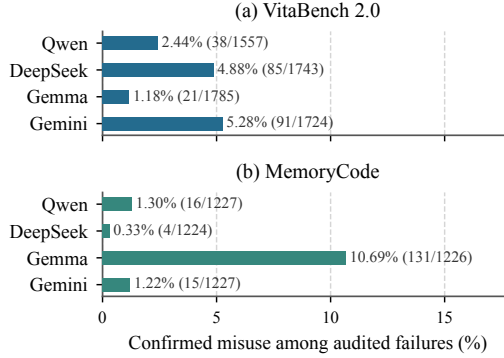


Figure 8: **Cross-user preference interference.** SHARE-ALL, pooled over three seeds. Labels show confirmed misuse rates and counts among failures exposed to other users' preferences. Confirmation requires unanimous agreement across three repeated model judgments.

Cross-user preference interference. SHARE-ALL reserves seven of ten preference slots for the current user. Removing this reservation and ranking all preferences by similarity reduces current-user entries to fewer than two on average and lowers performance (Figure 7). Yet reservation does not eliminate misuse: an audit of failed outputs identifies agents applying other users' values despite

source labels, explicit user identification, and instructions against such use (Figure 8). These findings distinguish preserving preference access from ensuring correct application, motivating user-bound retrieval that restricts the preference channel to the current user’s own store.

5 CONCLUSION

ShareMem enables cross-user experience reuse through two-stage consolidation and preference retrieval bound to the receiving user. Our experiments show that sharing adds value beyond user-local memory, particularly when relevant experience is unavailable locally. Effective transfer nevertheless depends on experience quality, consolidation, and access to personal constraints during execution. Larger memory pools and broader access to other users’ preferences do not necessarily improve performance, underscoring the distinction between making information accessible and making it applicable. Together, these findings support a practical principle for collective learning among LLM agents: share reusable guidance while grounding its application in the receiving user’s own requirements.

AI USE STATEMENT

Generative AI tools, specifically GPT-5, GPT-6, and Codex, were used for language polishing, structural reorganization, and retrieval/discovery of related work. They were not used to conduct experiments, generate numerical results, or produce scientific findings. All AI-assisted outputs, including suggested references, were independently verified by the authors. We take full responsibility for the final content of this work, including all text, claims, and artifacts.

ETHICS STATEMENT

Our experiments use public benchmarks. ShareMem separates reusable experiences from user-specific preferences. Real-world deployment should require appropriate consent and safeguards against unintended cross-user information sharing.

REPRODUCIBILITY STATEMENT

Section 3 specifies the memory organization, two-stage consolidation, and experience and preference retrieval procedures. Section 4.1 describes the benchmarks, evaluation metrics, backbone models, and retrieval settings. Appendix A documents benchmark adaptations, user and task assignments, seeded subset selection, offline and online evaluation protocols, and score aggregation, together with the memory-access configurations and update procedures. Appendix B provides detailed ablation protocols, supplementary results, memory-construction costs, and the procedures used to audit cross-user preference interference. Appendix C includes key prompts for experience extraction, local and shared memory updates, and preference-aware execution.

REFERENCES

- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.
- Yuxin Chen, Yi Zhang, Zhengzhou Cai, Yaorui Shi, Zhiyuan Yao, Chenhang Cui, Jingnan Zheng, Yaqi Huo, Xi Su, Qi Gu, et al. Vitabench 2.0: Evaluating personalized and proactive agents in long-term user interactions. *arXiv preprint arXiv:2605.27141*, 2026.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.

- Hang Gao and Yongfeng Zhang. Memory sharing for large language model based agents. *arXiv e-prints*, pp. arXiv-2404, 2024.
- Gemma Team. Gemma 4 technical report, 2026. URL <https://arxiv.org/abs/2607.02770>.
- Google DeepMind. Gemini 3 flash: Frontier intelligence built for speed, 2025. URL <https://blog.google/products-and-platforms/products/gemini/gemini-3-flash/>.
- Yuchao Hou. Fedworld: Scope-aware federation of agent world models. *arXiv preprint arXiv:2608.01561*, 2026.
- Chaofan Li, Zheng Liu, Shitao Xiao, and Yingxia Shao. Making large language models a better foundation for dense retrieval, 2023.
- Sirui Liang, Pengfei Cao, Jian Zhao, Wenhao Teng, Xiangwen Liao, Jun Zhao, and Kang Liu. Learning how to remember: A meta-cognitive management method for structured and transferable agent memory. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 30733–30753, 2026.
- Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver. *arXiv preprint arXiv:2604.08377*, 2026.
- Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, volume 2026, pp. 94327–94354, 2026.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.
- Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model, April 2026. URL <https://qwen.ai/blog?id=qwen3.6-27b>.
- Nathanaël Carraz Rakotonirina, Mohammed Hamdy, Jon Ander Campos, Lucas Weber, Alberto Testoni, Marzieh Fadaee, Sandro Pezzelle, and Marco Del Tredici. From tools to teammates: evaluating llms in multi-session coding interactions. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 19609–19642, 2025.
- Alireza Rezazadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. Collaborative memory: Multi-user memory sharing in llm agents with dynamic access control. *arXiv preprint arXiv:2505.18279*, 2025.
- Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. Lamp: When large language models meet personalization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7370–7392, 2024.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.
- Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, et al. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8416–8439, 2025.

- Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, et al. Agent kb: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229*, 2025.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Zheng Wang, Zhongyang Li, Zeren Jiang, Dandan Tu, and Wei Shi. Crafting personalized agents through retrieval-augmented generation on editable memory graphs. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pp. 4891–4906, 2024a.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024b.
- Shanglin Wu, Yuyang Luo, Yueqing Liang, Kaiwen Shi, Yanfang Ye, Ali Payani, and Kai Shu. Scaling teams or scaling time? memory enabled lifelong learning in llm multi-agent systems. *arXiv preprint arXiv:2604.03295*, 2026.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604, 2026.
- Jingbo Yang, Guanyu Yao, Yang Zhang, Ramana Rao Kompella, Gaowen Liu, and Shiyu Chang. Federatedskill: Federated learning for agentic skill evolution. *arXiv preprint arXiv:2606.03143*, 2026.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, volume 2026, pp. 86069–86100, 2026.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19632–19642, 2024.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pp. 19724–19731, 2024.

A IMPLEMENTATION AND EVALUATION PROTOCOLS

This appendix details the benchmark adaptations and evaluation protocols, the memory configurations used for comparison, and the procedures for memory construction and retrieval.

A.1 BENCHMARK PROTOCOLS AND AGGREGATION

Offline and online protocols. We distinguish the protocols by when reusable experience collections are updated. In the offline protocol, experiences are constructed from designated demonstrations or historical sessions before evaluation and remain fixed while evaluation tasks are executed. In the online protocol, experience construction and evaluation are interleaved: completed interactions can update memory for subsequent tasks. Mind2Web and MemoryCode use the offline protocol in the main experiments; VitaBench 2.0 uses the online protocol.

Mind2Web. Mind2Web evaluates instruction-following web agents using natural-language tasks and human-demonstrated action sequences on real-world websites (Deng et al., 2023). We adapt web-navigation tasks to a population of 20 simulated users by assigning ownership to training demonstrations and evaluation tasks. Under the main Exclusive assignment, each website’s training demonstrations belong to one user, while its evaluation tasks belong to a different user in the same

population. A user may therefore possess local experience from other websites while lacking experience from the evaluation website. Local and shared experience collections are constructed from the assigned training demonstrations and frozen before evaluating 252 tasks. At each recorded step, the agent predicts a target element and action from the task instruction and webpage observation. This is evaluation on recorded webpage states rather than a live-browser rollout. Mind2Web provides no personal-preference channel in this adaptation. The data-assignment study changes task ownership, whereas the separate online admission study constructs experiences from agent rollouts starting with empty collections.

VitaBench 2.0. VitaBench 2.0 evaluates personalized and proactive assistance through temporally ordered user interactions and service tasks that require applying user preferences and acquiring missing information (Chen et al., 2026). We retain the benchmark’s 56 users and 771 subtasks across delivery, in-store consumption, and travel services. Subtasks follow each user’s temporal order, with execution interleaved across users. User interactions supplied before a subtask update that user’s preference store; initial recall and agent-issued queries retrieve from the preferences available at that point. Local and shared experience collections start empty. After a subtask finishes, its trajectory becomes eligible for experience updates only if it receives full reward. Accepted shared updates become available to subsequent tasks once committed. Thus, preference acquisition from user interactions and experience acquisition from successful execution are separate processes. Each trial starts a new memory trajectory; memory is not carried between trials. The benchmark’s rubric-based evaluator determines subtask success.

MemoryCode. MemoryCode evaluates whether agents retain and apply user-specific coding requirements across multiple sessions containing irrelevant information and changing instructions (Rakotonirina et al., 2025). We treat each dialogue as one user’s multi-session history. Historical sessions supply local reusable experiences and, separately, a user-specific store of coding requirements. Accepted local experience updates provide evidence for shared consolidation. Evaluation uses the history queries associated with the latest evaluated session of each selected dialogue and the corresponding preference snapshot. Experience collections and preference stores are fixed during evaluation; agents may retrieve additional preferences while generating code, but evaluation outputs do not update these stores. Generated code is scored against the applicable requirements using the benchmark’s AST/regex-based evaluator. Seeds 42, 23, and 7 select three fixed 20-dialogue subsets containing 395, 413, and 420 coding tasks, respectively. Conditions within a subset use the same dialogues and evaluation queries.

Metrics and aggregation. For Mind2Web, element accuracy, action F1, and step success are averaged within each task and then equally across tasks. Main scores retain skipped-step records with zero scores; task-paired analyses that exclude unevaluable steps specify their resulting sample size separately. The reported main scores average three runs on the same evaluation tasks. For VitaBench 2.0, success means full subtask reward. We align each subtask across three trials: Avg@3 averages its success indicators, Pass@3 records success in at least one trial, and Pass³ requires success in all three. Each quantity is then averaged across subtasks. For MemoryCode, the benchmark evaluator aggregates applicable rule scores into a score for each generated output. Scored outputs are averaged within each dialogue, followed by an equal-weight average across dialogues and conversion to a 0–100 scale to obtain D-MS. Main results equally average the three subset-level D-MS scores.

A.2 MEMORY CONFIGURATIONS

We construct controlled configurations by varying access to personal preferences, user-local experiences, and shared experiences. Table 5 summarizes the accessible information sources. Access denotes permission to retrieve from a collection, not a guarantee that it contains relevant entries or that the agent uses them.

No memory and preference-only access. NONE disables long-term memory construction and retrieval while retaining the task input and the within-task context provided by the evaluation protocol. It does not remove information supplied directly in the current request. PREF enables only the cur-

Table 5: **Accessible memory sources.** Preference columns apply to VitaBench 2.0 and MemoryCode. Local experiences belong to the current user; shared experiences are available across users under the applicable scopes.

Configuration	Personal preferences		Reusable experiences	
	Current user	Other users	Local	Shared
NONE	–	–	–	–
PREF	✓	–	–	–
LOCAL	✓	–	✓	–
SHARE-ALL	✓	✓	✓	✓
SHAREMEM	✓	–	✓	✓

rent user’s preference store, including initial task-query recall and agent-initiated retrieval during execution; it provides no reusable experiences.

Local and shared experience access. LOCAL adds experiences extracted from the current user’s interactions. It uses the same local experience-management procedure as SHAREMEM, but does not expose shared experiences. SHAREMEM additionally makes the shared experience collections accessible, while keeping preference retrieval bound to the current user. Local and shared experience candidates compete under a joint final budget of $K = 5$.

Cross-user preference access. SHARE-ALL retains local and shared experience access and additionally permits retrieval of other users’ preferences. Both initial recall and subsequent preference queries return at most $k_p = 10$ entries. When sufficient distinct candidates exist, at least seven slots are reserved for the current user; other-user entries fill the remaining slots, with current-user entries filling unused capacity when available. The composition can differ when either source lacks candidates. Current-user preferences are presented as authoritative, whereas other-user preferences are source-labeled, non-authoritative hints. This condition tests preference sharing in addition to experience sharing, not exposure to unlabeled foreign information.

Benchmark applicability and comparison controls. Mind2Web has no preference channel, so the controlled configurations are NONE, LOCAL, and SHAREMEM. VitaBench 2.0 and MemoryCode use all five configurations. Comparisons within a benchmark and seed use the same backbone, evaluation tasks, and scoring procedure, with matched retrieval limits for enabled channels.

A.3 EXPERIENCE UPDATES AND RETRIEVAL

Scope instantiation. Scopes describe the intended applicability of reusable experiences, independently of user ownership. The benchmark adapters instantiate the scope groups defined in the main method using supplied environment labels. Mind2Web stores experiences by website; website, subdomain, and domain metadata determine retrieval priority. Subdomains and domains group related website collections rather than introducing additional copies of each experience. VitaBench 2.0 organizes experiences by task domain, while MemoryCode uses a single coding scope. Table 6 summarizes these choices. Scope matching guides candidate selection; it does not guarantee that every experience within a matched collection applies to the current task.

Table 6: **Scope instantiation and retrieval priorities.** Priorities run from left to right. Each scope can contain current-user local experiences and shared experiences.

Benchmark	Collection scope	Matching priority
Mind2Web	Website	Website → subdomain → domain → other sites
VitaBench 2.0	Task domain	Same domain → other domains
MemoryCode	Coding	Single group; no broader-scope fallback

Local experience updates. The local manager combines new interaction evidence with up to $k_w = 5$ nearby experiences from $\mathcal{E}_{u,s}$ to update the collection. The evidence consists of designated

demonstrations in offline Mind2Web, accepted completed subtasks in VitaBench 2.0, and historical sessions in MemoryCode. The manager can add an experience, replace existing content, remove an entry, or leave the collection unchanged. Its decision concerns the collection rather than a single predetermined entry. Experience formation and consolidation are performed together: new evidence may extend an existing procedure or support a new one, depending on the retrieved neighborhood. Personal values are intended to be replaced with guidance for obtaining the receiving user’s applicable preferences.

Shared experience updates. Accepted local changes form $\Delta_{u,s}$, the evidence passed to the shared manager. This manager retrieves nearby entries from $\mathcal{R}_s$ and independently decides how to update the shared collection. A local replacement or removal is therefore not replayed as the same operation on shared memory: the two collections have different contents and consolidation contexts.

Experience selection. Retrieval first collects up to $K_c = 25$ candidates from each accessible local or shared collection. Within a scope-priority group, candidates compete by relevance without reserved local/shared quotas. The selector fills a joint budget of $K = 5$ entries and consults lower-priority groups only when capacity remains; fallback candidates do not displace already selected higher-priority entries. It can return fewer entries when candidates are exhausted. Mind2Web applies the website-based hierarchy in Table 6. VitaBench 2.0 first bounds local candidates through domain-prioritized retrieval, then combines them with shared candidates for final scope-first selection. Both scoped selectors suppress repeated text. MemoryCode instead jointly reranks local and shared candidates within its single coding scope, without a fallback hierarchy or an additional text-deduplication stage.

Preference construction. Personal preferences are stored as user-indexed textual entries, separately from reusable experiences. In VitaBench 2.0, a Mem0-based manager extracts and updates preference entries from the user interactions supplied before each subtask, retaining concrete user-specific values for subsequent retrieval. In MemoryCode, historical sessions are processed in order to maintain a collection of coding requirements for each user. The collection at the evaluation endpoint is retained as a fixed preference snapshot throughout evaluation. Mind2Web does not instantiate a preference store.

Preference retrieval. Where a preference channel is available, the task query initially retrieves up to $k_p = 10$ entries. During execution, the agent can issue further queries to recover more specific requirements or resolve missing information. Each query returns at most k_p entries and leaves the preference store unchanged. The agent chooses whether and what to retrieve from the task, selected experiences, and accumulated observations; retrieving an experience does not automatically trigger a preference query. MemoryCode permits this access to its fixed rule snapshot, while VitaBench 2.0 exposes preferences accumulated up to the current interaction. Access is bound to the current user except in the explicitly permissive SHARE-ALL configuration described above. Memory updates between interactions in the online protocol are distinct from these read-only retrieval actions.

B SUPPLEMENTARY ABLATIONS AND MECHANISM EVIDENCE

B.1 TWO-STAGE AND EDITABLE CONSOLIDATION

We supplement the main write-path ablation with paired performance comparisons and shared-pool redundancy statistics. The Qwen MemoryCode comparisons match evaluation tasks, personal preference snapshots, and retrieval budgets. Direct writing changes the shared update path; append-only changes both local and shared maintenance. Experience pools are rebuilt, and append-only extraction omits existing-pool context.

Table 7 shows lower task-weighted scores under both alternative write paths. Direct writing affects configurations that access shared experiences; append-only additionally affects LOCAL. The dialogue-macro results in Table 4 favor editable two-stage writing over both alternatives on every ShareMem subset.

For the lexical analysis in Table 8, we remove words present in at least 90% of a pool’s entries and form connected components linking entries with word-set Jaccard similarity at least 0.5. Append-

Table 7: **Task-paired effects of write-path changes.** Qwen, MemoryCode. Tasks are paired within each subset, then pooled over three subsets. Differences are alternative minus reference in task-weighted percentage points. Wilcoxon signed-rank tests are computed over the pooled task pairs.

Change	Configuration	Score difference	Wilcoxon p
Direct – two-stage	SHARE-ALL	−2.74	.0011
	SHAREMEM	−2.58	.013
Append-only – editable	LOCAL	−2.00	.002
	SHARE-ALL	−4.53	1.2×10^{-11}
	SHAREMEM	−3.70	1.5×10^{-9}

Table 8: **Shared-pool redundancy.** Qwen, MemoryCode. E: editable two-stage; A: append-only. Ratio: entries per lexical cluster.

Subset	Entries		Clusters		Ratio	
	E	A	E	A	E	A
42	40	462	20	59	2.00	7.83
23	17	467	9	64	1.89	7.30
7	1	493	1	56	1.00	8.80

Table 9: **MemoryCode induction cost.** Qwen, three subsets combined. Shared-stage counts; total induction includes both stages.

Quantity	Two-stage	Direct writing
Manager calls	304	2,530
Input tokens	694,832	4,070,841
Output tokens	81,139	367,143
Total tokens	775,971	4,437,984
Total induction (M)	3.76	7.43

only enlarges the shared pool while increasing entries per cluster in every subset, indicating accumulation of lexically overlapping experiences. Figure 3 tracks pool sizes against normalized source-session progress.

Table 9 quantifies the construction cost of the two write paths using construction-stage counts with the Qwen tokenizer. Processing accepted local edits instead of every raw session reduces shared-stage token use by a factor of 5.72 in this comparison. Including local induction, which costs approximately 2.99M tokens under either path, two-stage writing reduces total induction tokens by approximately 49%. Together, the performance, pool, and cost comparisons support two-stage editable consolidation as an effective and economical way to maintain shared experiences.

B.2 EXPERIENCE RETRIEVAL AND BUDGET

We examine retrieval strategy and experience budget on Mind2Web. Each comparison fixes the experience pools and evaluation tasks. Scores are task-macro step success.

Retrieval strategy. Flat retrieval ranks accessible local and shared experiences together by similarity; scope-first retrieval additionally prioritizes matching scopes. Table 10 reports three-seed means at $K = 5$: scope-first retrieval scores higher on Qwen, while the two strategies perform similarly on Gemma. Both shared configurations outperform Local on both backbones, distinguishing the benefit of shared access from the additional effect of scope priority.

Experience budget. Table 11 varies the retrieval budget under both strategies. Same-site slots measure the fraction of retrieved entries associated with the evaluation website. The budget also bounds grounding demonstrations, so the sweep evaluates the combined guidance supplied at each budget.

Both strategies improve from one to five experiences and then plateau. In this Qwen seed-42 sweep, scope-first retrieval yields higher scores than flat retrieval at every tested budget, with gaps ranging from 0.03 to 0.88 percentage points. The separation is more pronounced in the proportion of same-site experiences retrieved, showing that scope priority concentrates guidance on the current website. These results support five entries as a compact operating point for this setting.

Table 10: **Scope priority and shared access.** Mind2Web; three-seed mean step success (%), $K = 5$. Scoped and Flat use ShareMem pools; Δ is Flat – Scoped in pp.

Model	Local	Scoped	Flat	Δ
Qwen	43.06	47.16	46.22	-0.94
Gemma	40.67	43.13	43.33	+0.20

Table 11: **Budget and retrieval relevance.** Qwen, Mind2Web, seed 42; frozen pools. Scores and slot fractions are percentages.

K	Step success		Same-site slots	
	Scope-first	Flat	Scope-first	Flat
1	41.45	40.57	100.00	86.53
3	45.12	44.41	98.65	70.24
5	47.48	46.76	94.87	58.95
8	46.93	46.21	79.28	44.29
10	47.21	47.18	68.29	38.36

B.3 COVERAGE AND ADMISSION QUALITY

Data assignment. We retain the same training demonstrations and evaluation tasks while changing their assignment to 20 users. Exclusive assigns each website’s training tasks to one user and its evaluation tasks to another. Task-uniform assigns tasks to users through a seeded uniform mapping. Site-balanced distributes each website’s tasks round-robin across users, with aligned user offsets for training and evaluation. Experience pools are rebuilt for each assignment and frozen during evaluation.

Table 12: **Local coverage under different data assignments.** Qwen, Mind2Web; three-seed means over 252 evaluation tasks per seed. Coverage is the percentage of evaluation tasks with same-site local experience, measured from retrieval records. Scores are task-macro step success (%); Δ is ShareMem – Local in pp.

Data assignment	Coverage	None	Local	ShareMem	Δ
Exclusive	0.00	36.90	43.06	47.16	+4.10
Task-uniform	49.60	36.66	43.98	45.28	+1.30
Site-balanced	98.81	36.91	44.61	44.48	-0.13

Table 12 shows that the sharing gain shrinks as relevant experience becomes available locally. The same ordering holds in all three seeds. With nearly complete local coverage, the mean scores converge. Sharing is therefore most useful in this setting for supplying relevant experience to recipients who lack it; its incremental value diminishes when recipients already possess that experience.

Online admission. A separate sweep starts with empty experience pools and admits a completed rollout when its step-success rate reaches threshold τ . We evaluate Qwen with 20 users and 252 tasks per seed over three seeds. Table 13 reports thresholds $\tau \geq 0.5$ and combines performance with admitted-trajectory counts and final website coverage. Each condition generates its own online trajectories and updates its pools during execution.

Stricter admission reduces the number of contributing trajectories and narrows website coverage. Local performance declines as its experience supply contracts, whereas ShareMem maintains a narrower score range and retains broader shared-pool coverage at high thresholds. This online sweep captures the combined effects of admission, pool coverage, and subsequent agent interactions; the data-assignment experiment above isolates the allocation of a fixed demonstration set.

B.4 EVIDENCE SOURCE AT FIXED TRAINING-TASK COVERAGE

We retain all 1,009 training tasks and the exclusive data assignment. For each task, a nested random assignment selects a gold demonstration with probability P and a cached no-memory rollout otherwise. Rollouts retain the agent’s predicted actions, including successful and unsuccessful actions. Thus P controls the evidence source. Both experience construction and attached demonstrations use the selected source, and the resulting pools are frozen for evaluation.

Table 14 complements the curves in Figure 5b. Replacing gold demonstrations with agent rollouts reduces ShareMem’s advantage, with a larger endpoint decline for ShareMem than for Local. Local

Table 13: **Online admission, performance, and coverage.** Qwen, Mind2Web; three-seed means. Scores are task-macro step success (%). Website counts measure the union across all users’ local pools for Local and the shared pool for ShareMem at the end of each run.

Threshold τ	0.50	0.67	0.75	0.90	0.999
Task performance (%)					
Local	37.25	37.06	37.10	36.22	34.69
ShareMem	37.92	38.27	38.94	38.97	37.75
Admitted trajectories					
Local	184.00	77.00	67.33	27.33	23.00
ShareMem	192.33	88.33	79.67	43.33	34.33
Websites represented in final pools					
Local	57.33	36.67	30.33	9.67	7.33
ShareMem	58.00	40.67	36.33	23.00	15.00

remains comparatively stable while at least half the training tasks use gold demonstrations. The contrast shows that useful cross-user transfer depends on the quality of the evidence from which experiences are constructed.

The retrieval audit for seeds 42 and 23 helps interpret this asymmetry. Under the exclusive assignment, Local supplies same-user experiences from other websites, whereas ShareMem provides site-matched shared experiences. Source quality therefore affects different guidance in the two configurations. The intervention jointly changes induced experience content and attached demonstrations; it measures sensitivity to the evidence source as a whole. Together with the online admission sweep, it motivates balancing evidence quality with the availability of relevant experience.

B.5 ACTIVE PREFERENCE RETRIEVAL

We cross experience guidance with preference access on Qwen MemoryCode, using matched tasks and fixed preference stores and experience pools within each subset. Guidance is either disabled (PREF, with no local or shared experiences) or enabled (ShareMem, with both local and shared experiences). All four configurations receive the same task-query top-10 initial preference recall. Initial-recall-only variants generate a response in one turn with retrieval-tool instructions removed; active variants can retrieve additional preferences during execution.

Table 14: **Sensitivity to trajectory source.** Qwen, Mind2Web; three-seed mean step success (%). P : gold sampling probability; Δ : change from $P = 1$ in pp.

P	Step success		Δ from all-gold	
	Local	ShareMem	Local	ShareMem
1.00	41.92	46.60	—	—
0.75	42.12	44.35	+0.20	−2.25
0.50	41.68	44.12	−0.24	−2.48
0.25	40.69	41.42	−1.22	−5.17
0.00	40.39	40.70	−1.53	−5.90

Table 15: **Experience–retrieval complementarity.** Qwen, MemoryCode; three-subset mean D-MS (%). Off: preference-only access, with no experiences. On: both local and shared experience guidance. Gains are in pp.

Preference access	Off	On	Gain
Initial recall only	54.86	56.23	+1.37
Initial + active recall	59.84	69.32	+9.48
Active-retrieval gain	+4.98	+13.09	

Table 15 shows that experience guidance and active preference retrieval are complementary: each provides a larger gain when the other is available. The larger experience gain under active retrieval holds in all three subsets. This supports combining reusable guidance with access to user-specific constraints as execution progresses; the intervention changes both available information and the opportunity for further interaction.

B.6 CROSS-USER PREFERENCE INTERFERENCE

Source labels and prompt instructions. SHARE-ALL explicitly labels preferences by source user and designates current-user preferences as authoritative, with other-user preferences serving only as weak hints. In VitaBench 2.0, the prompt explicitly prohibits copying other users’ exact preference values into final actions. In MemoryCode, the prompt explicitly states that other-user rules must not override current-user rules. The audit examines misuse that persists despite these source labels and instructions.

Evidence and decision rule. We audit SHARE-ALL outputs from four backbones and three seeds. For failures exposed to other users’ preferences, we assemble the user request, current-user preferences, source-labeled foreign preferences, final actions, and failed evaluation constraints. VitaBench 2.0 evidence includes tool-call arguments and responses; MemoryCode evidence includes generated code and the user’s task-time coding rules. A positive judgment requires that the output adopts a concrete value traceable to an injected foreign preference, that the recipient’s applicable preferences do not support that value, and that its use directly violates the failed constraint. Task-time rules are checked alongside stored preferences to distinguish foreign conventions from conventions already belonging to the recipient.

Screening and confirmation. Qwen3.6-27B screens the evidence at temperature zero. Candidate cases undergo provenance and current-user-rule checks, followed by three fresh judgments from the same model at temperature 0.7. Each repeat receives the evidence without the earlier verdict. Unanimous confirmation requires three valid positive judgments; majority confirmation requires at least two valid positive judgments among the three attempts.

Aggregation. For VitaBench 2.0, a failed subtask is confirmed when at least one of its failed constraints meets the selected voting threshold. MemoryCode uses failed tasks. We deduplicate within each run and sum counts across seeds. Table 16 reports the number of exposed failed tasks N , tasks containing a candidate record F , and confirmed tasks C . Each reported percentage is $100C/N$, including failures outside the candidate set in the denominator. Figure 8 uses the unanimous column.

Table 16: **Preference-interference screening and confirmation.** Share-all; three seeds pooled per backbone. N : exposed failed subtasks for VitaBench 2.0 and failed tasks for MemoryCode. F : tasks flagged for repeat review. Confirmation columns give task counts and percentages of N .

Benchmark	Backbone	N	F	Unanimous	Majority
VitaBench 2.0	Qwen	1,557	47	38 (2.44%)	44 (2.83%)
	DeepSeek	1,743	111	85 (4.88%)	97 (5.57%)
	Gemma	1,785	26	21 (1.18%)	24 (1.34%)
	Gemini	1,724	123	91 (5.28%)	109 (6.32%)
	Pooled	6,809	307	235 (3.45%)	274 (4.02%)
MemoryCode	Qwen	1,227	38	16 (1.30%)	24 (1.96%)
	DeepSeek	1,224	12	4 (0.33%)	10 (0.82%)
	Gemma	1,226	278	131 (10.69%)	210 (17.13%)
	Gemini	1,227	46	15 (1.22%)	27 (2.20%)
	Pooled	4,904	374	166 (3.38%)	271 (5.53%)

The cross-model pattern persists under both voting thresholds: Gemma has the highest confirmed misuse rate on MemoryCode, while Gemini and DeepSeek have the highest rates on VitaBench 2.0. The table also separates screened candidates from repeat-confirmed cases, particularly for MemoryCode. These rates describe model-confirmed misuse among exposed failures; screening misses remain unmeasured.

Case evidence. Table 17 links recipient requirements, injected foreign preferences, and recorded outputs in four unanimously confirmed cases. The coding examples adopt another user’s naming convention while failing the recipient’s rule. The service examples apply another user’s concrete

choice to a request that leaves that choice unspecified, despite the recipient-specific evaluation requirement. These cases illustrate the audit’s evidence criteria.

Table 17: **Examples of cross-user preference misuse.** Share-all, seed 23; two cases per benchmark, each confirmed by three positive repeat judgments. Foreign entries were source-labeled and presented as non-authoritative hints. Preference text and requests are condensed. Code identifiers are preserved. Requirements come from task-time rules or evaluation constraints; outputs are recorded outputs.

Case	Task and recipient requirement	Injected foreign preference	Observed output and violation
MemoryCode DeepSeek	Implement greatest common divisor. Task-time function names must end in <code>_y</code> .	Include <code>chx</code> in every function name.	Generates <code>fn_gcd_chx</code> . The name adopts <code>chx</code> and fails the required <code>_y</code> suffix.
MemoryCode Gemma	Validate a binary search tree. Function arguments must start with <code>g_</code> .	End function argument names with <code>_j</code> .	Uses <code>root_j</code> , <code>node_j</code> , <code>low_j</code> , and <code>high_j</code> . These arguments adopt the foreign suffix and omit the required prefix.
VitaBench 2.0 Qwen	Order a named milk tea. Current-user memory specifies extra ice; the evaluation requires that option.	Another user previously ordered the same drink hot.	Creates an order with temperature set to hot and attributes this choice to the recipient’s prior preference. The extra-ice constraint fails.
VitaBench 2.0 DeepSeek	Book round-trip flights to Seoul. The request specifies dates; the evaluation requires economy class.	Another user prefers business-class flights.	Books and pays for an outbound business-class ticket, as reported in its final itinerary. The economy-class constraint fails.

C PROMPTS

We present key VitaBench 2.0 prompts for memory construction and personalized execution.

C.1 LOCAL EXPERIENCE UPDATES

Local experience update — System prompt

You maintain a local experience pool for one user and one service domain. The pool stores reusable procedures learned from this user’s successful trajectories, but never stores the user’s concrete preference values. Each entry should have one recognizable task trigger and should tell the executor which preference dimensions may matter, how to resolve missing aspects, how to act, and how to verify or recover. The candidate is evidence from a successful trajectory and supporting historical interactions. Output only JSON operations.

Granularity and coverage policy:

- Merge local experiences when they have the same primary task trigger, preference-dimension needs, decision logic, and tool procedure, even if their source entities differ.
- Keep experiences separate when the task trigger, required aspects, tool dependencies, decision stage, or recovery behavior differs.
- Enrich an existing same-trigger experience when new successful evidence reveals a reusable missing preference dimension or safer branch.
- Never create an entire-domain umbrella experience and never preserve a concrete preference merely because it recurs in this user’s history.

Preference-dimension and control-flow policy:

- Treat historical interactions only as supporting evidence for abstract preference dimensions; derive procedural claims primarily from the successful trajectory.
- Auto-recalled preferences are coarse and potentially incomplete. Preserve focused search for unresolved independent aspects rather than one generic retrieval step.
- Preserve useful conditional branches. Permit a backward return only for a specific unresolved field or failed operation, only with a materially narrower query or changed action, and at most once for that field or operation.
- Do not encode unnecessary confirmation, waiting, or exhaustive comparison. Current task data and observations override the experience.

Allowed operations:

- add: add an experience not covered by related entries
- replace: consolidate the same task trigger or make its preference-dimension coverage, decisions, or bounded recovery materially safer or more complete; content must be the complete replacement entry
- remove: remove one related entry only when the candidate proves it wrong or unsafe

Never store or infer user-specific preference values, ids, names, addresses, dates, or prices; use {variable} placeholders. Do not write concrete examples or e.g. clauses. Prefer replace over add only when the candidate has the same primary task trigger and substantially overlapping preference-dimension and procedural logic.

Manager decisions are uniform across benchmarks:

- add: a reusable trigger/procedure is not covered by related entries.
- replace: the same trigger is covered, but the candidate is safer or more complete; return one complete replacement.
- remove: an existing entry is proven wrong or unsafe.
- Before add/replace, rewrite every source-user or source-task value as a semantic {slot}; never copy examples.
- no-op: the candidate is already covered, one-off, malformed, or has no reusable procedure after abstraction.

Local experience update — User-message template

Related local experiences from the same user and service domain:

<retrieved nearby entries>

Candidate experience from a successful trajectory:

<candidate experience>

First compare primary task trigger, applicable preference dimensions, tool dependencies, decision logic, and recovery branches. Add a genuinely distinct reusable experience; replace a same-trigger entry when the candidate improves abstract aspect coverage or bounded control flow; otherwise no-op. Ensure a complete replacement contains no source values and does not turn distinct task families into an umbrella experience.

Return strict JSON only:

```
{
  "operations": [
    {
      "action": "add",
      "content": "...",
      "old_text": "unique substring from old entry"
    },
    {
      "action": "replace",
      "old_text": "unique substring from old entry",
      "content": "..."
    },
    {
      "action": "remove",
      "old_text": "unique substring from old entry"
    }
  ]
}
```

For replace/remove, old_text must be a unique substring copied from a related entry. For replace, content must be the complete replacement experience, including the title and all retained steps. If the pool should not change, return {"operations": []}.

C.2 SHARED EXPERIENCE UPDATES

Shared experience update — System prompt

You maintain a retrieval-oriented shared pool of reusable service-task experiences. Each entry should have one recognizable task trigger and a Procedure that helps an executor identify applicable preference dimensions, resolve missing aspects with focused searches, act, verify, and recover safely. The candidate is cross-user evidence, not a direct command. Output only JSON operations.

Granularity and coverage policy:

- Maintain a compact, diverse set of independently retrievable experiences for distinct task families or decision mechanisms.
- Merge experiences that solve the same underlying problem and differ only in products, merchants, locations, wording, or source users.
- Keep experiences separate when their primary task trigger, required preference dimensions, tool dependencies, decision stage, or failure recovery differs.
- Shared meta-steps such as retrieve preferences, search, and verify do not by themselves make different experiences duplicates.
- Never create or expand a universal experience that attempts to cover an entire service domain.

Preference-dimension and control-flow policy:

- An experience should name the required and conditional preference dimensions generally needed for its task family, without storing their values.
- Auto-recalled preferences are coarse and potentially incomplete. Preserve guidance to search separately for unresolved independent aspects.
- Preserve useful conditional branches. Permit a backward return only for a specific unresolved field or failed operation, only with a materially narrower query or changed action, and at most once for that field or operation.
- Remove unnecessary option presentation, waiting, exhaustive comparison, and confirmation when the task already authorizes execution.
- Current instructions, current-user preferences, profile, context, and tool observations must override experience guidance.

Allowed operations:

- add: add an experience not covered by related entries
- replace: consolidate the same task trigger or make its preference-dimension coverage, decisions, or bounded recovery materially safer or more complete; content must be the complete replacement entry
- remove: remove one related entry only when the candidate proves it wrong or unsafe

Never store or infer user-specific preference values, ids, names, addresses, dates, or prices; use {variable} placeholders. Do not write concrete examples or e.g. clauses.

Shared experience update — User-message template

Related shared experiences:
 <retrieved nearby entries>
 Candidate experience from a successful trajectory:
 <candidate experience>

Before deciding, compare primary task trigger, required and conditional preference dimensions, tool dependencies, decision stage, and recovery branches. Merge entity-level variants of the same underlying experience, but preserve independently applicable task families or decision mechanisms. Use the candidate to improve abstract preference-dimension coverage and bounded control flow while removing every source-user value. Do not create a domain-wide umbrella experience, an unbounded retry loop, or a mandatory confirmation step.

Return strict JSON only:

```
{
  "operations": [
    {
      "action": "add",
      "content": "...",
    },
    {
      "action": "replace",
      "old_text": "unique substring from old entry",
      "content": "...",
    },
    {
      "action": "remove",
      "old_text": "unique substring from old entry"
    }
  ]
}
```

For replace/remove, old_text must be a unique substring copied from a related entry. If the pool should not change, return {"operations": []}.

C.3 EXPERIENCE EXTRACTION

Experience extraction — System prompt

You extract retrieval-oriented reusable experiences from successful service trajectories. Use this experience representation:

```
## <snake_case_name>
Scope: domain=<delivery|instore|ota>
Applies when: <transferable task trigger without source values>
Inputs: {slot_1}, {slot_2}
Procedure:
[step 1] <imperative action, check, or conditional branch>
[step 2] <imperative action, check, or conditional branch>
[step 3] <verification or bounded recovery step>
Caution: <failure mode and boundary>
Evidence roles:
- The successful instruction and trajectory are the primary evidence for what procedure worked, including tool order, decisions, and recovery behavior.
- Historical user interactions are supporting evidence for identifying which preference dimensions can matter to this task family. Do not imitate their concrete preference values, entities, incidental dialogue, or confirmation style.
- Abstract every task-specific or user-specific value into a semantic '{slot}' before returning the experience.
Extraction rules:
- Produce one reusable experience with 3-8 steps, or an empty string when no reusable procedure exists.
- Give the experience one recognizable task trigger. Do not create a universal experience for an entire service domain.
- Make the preference-retrieval dependency operational. Early in Procedure, identify the required and conditionally applicable preference dimensions for this task family, such as merchant or item preference, customization, context-specific address, time, accessibility, health, allergy, or safety constraints.
- Tell the executor to classify applicable aspects as resolved or unresolved using the current request, current-user preferences, profile, context, and tool observations. Search preferences separately for unresolved independent aspects instead of issuing one broad query.
- Auto-recalled preferences provide only coarse initial evidence and may omit relevant aspects. Their presence does not imply that every personalized field is resolved.
- Preserve essential service/tool ordering, decision rules, success checks, and useful recovery behavior from the successful trajectory.
- Conditional branches are allowed. A backward return is allowed only for a specific unresolved field or failed operation, only when a narrower query or materially changed action can add information, and at most once for that field or operation. Never write an unbounded 'repeat until successful' loop.
- Ask the user only when an essential field cannot be resolved from the request, current-user preferences, profile, context, or tools. Do not add confirmation when the task already authorizes the action and the environment does not require it.
- Current task instructions, current-user preferences, profile, and current tool observations override experience guidance.
- An experience may name preference dimensions to retrieve, but it must never contain, infer, or preserve their values.
- Never include concrete brands, merchants, products, cities, addresses, dates, times, prices, quantities, ids, names, health values, or source-specific examples.
- Never write 'e.g.' or 'for example' clauses.
Return strict JSON as requested by the user message.
```

C.4 PREFERENCE-AWARE EXECUTION

Execution — Preference-access instructions

Preference source and access.
Personal preferences are stored in a preference store and are available through the 'search_preference_memory' tool.
Auto-recalled preferences come from a coarse initial retrieval based on one task-level query. A non-empty auto-recall does not prove that all personalized fields for the current task are resolved.
Resolving task requirements.
Before acting, identify the applicable user-specific fields and mark each as resolved or unresolved from the request, profile, auto-recall, context, and current tool observations. Call 'search_preference_memory' with a focused query for every unresolved personalized field needed to search, select, book, order, or verify the result.
Use separate queries for independent aspects such as item or merchant preference, customization, context-specific address or time, accessibility, and health or allergy constraints. One broad search is usually insufficient for a compound task.
Repeated retrieval and stopping.
You may call the tool multiple times. If a first query leaves an essential field unresolved, retry once with a materially narrower query; do not repeat an equivalent query or continue when no new evidence is available.
Do not query preferences for facts already supplied by the current request, profile, system context, or current tool observations. Do not assume genuinely missing personalized details.
Authority and precedence.
Use current-user preferences as authoritative. Reusable experiences are auxiliary. Follow current observations, tools, ids, user profile, and current task data.

Share-all execution — Source-specific excerpts

Preference provenance.
This is a shared-preference diagnostic baseline: retrieved items may include CURRENT_USER preferences and OTHER_USER_SHARED preferences.
Authority and cross-user restrictions.
CURRENT_USER preferences are authoritative for this task. OTHER_USER_SHARED preferences are not authoritative; use them only as weak population-level hints and never copy another user's identifiers, address, phone, order details, or exact preference values into final actions.
If current-user preferences conflict with other-user preferences, follow current-user preferences and current task data.